# Detecting Toxic Comments in Online Discussions: A Pilot Comparison of Supervised Models

Moaz Allam, Kareem Elhenawy
Department of Computer Science and Engineering, The American University in Cairo
Moaz_Allam@aucegypt.edu, karimelhenawy@aucegypt.edu
*CSCE 3602 / DSCI 3415 Class Project, Phase 3: Pilot Study of Supervised Models*

## ABSTRACT

We compare the supervised models of the course on the Civil Comments toxicity data: linear and logistic regression, the perceptron, Naive Bayes, three decision trees (ID3, CART, C4.5), a random forest, linear and RBF-kernel support vector machines (SVMs), a multi-layer perceptron (MLP), AdaBoost and k-nearest neighbours. Each predicts six toxicity attributes from 33 tabular features and a 100-dimension text embedding, either averaged Word2Vec or Doc2Vec; PV-DBOW Doc2Vec matches Word2Vec and beats PV-DM. Every setting is chosen on validation and reported on test with all Lecture 2 metrics. The RBF SVM has the best F1 at its default cut (toxicity 0.407), and the MLP ranks comments best (ROC-AUC 0.882). A deeper study of the top three (RBF SVM, MLP, logistic regression) with cross-validation, repeated seeds, label thresholds of 0.4 and 0.6, decision-cut sweeps and identity-group bias metrics shows that most of the difference at the default cut comes from how many comments each model flags: with a cut chosen on validation, the MLP reaches test F1 0.487 against 0.410 for the SVM. Results on the four rare labels are noisy for every model, and all three models rank clean comments that mention some identity groups too high. We choose the MLP for the rest of the project.

**KEYWORDS** toxic comment classification, model comparison, support vector machine, neural network, class imbalance, result variability

## 1 INTRODUCTION

Our project builds a moderation aid: a model that reads a comment and scores it on six toxicity attributes so that a moderator can read the worst comments first. Phase 1 chose the Civil Comments dataset and framed the task as multi-label classification: a comment is positive for an attribute when at least half of its raters said so [1]. Phase 2 cleaned the data and turned each comment into 133 scaled features [2].

This phase is a pilot study. We train every supervised model covered in the course on the same features and splits, describe how each was set up and tuned, and comment on how well it fits the problem (Section 3). We then take the three best models, compare them in more depth, and measure how much their results move when the data sample, the random seed, the label definition, the decision cut or the group of people a comment mentions changes (Section 4). Section 5 chooses one technique for the rest of the project.

Two changes from the earlier plans apply here. Following the feedback on Phase 2, the text is also encoded with Doc2Vec, which learns one vector for the whole comment instead of averaging word vectors. And the Phase 1 plan to fine-tune a transformer is dropped, because the project is limited to the models taught in the course.

## 2 EXPERIMENTAL SETUP

### *2.1 Task, data and features*

**Labels.** Six binary targets: toxicity, insult, identity_attack, obscene, threat and sexual_explicit, each positive when its score is at least 0.5. One model is trained per label. **Splits.** The cleaned Phase 2 splits: 1,777,414 training, 97,320 validation and 97,320 test comments. Positives in training: toxicity 8.0%, insult 5.9%, identity_attack 0.75%, obscene 0.53%, sexual_explicit 0.26% and threat 0.24%.

**Features.** Each comment has 33 scaled tabular features from Phase 2 (text statistics, a reply flag, publication one-hot) and 100 text dimensions. We compare three versions of the text dimensions: the mean of the comment's Word2Vec vectors from Phase 2 [6], and the two Doc2Vec models [4], PV-DM and PV-DBOW. PV-DM predicts a word from its neighbours plus a paragraph vector; PV-DBOW predicts the comment's words from the paragraph vector alone, ignoring word order. Both Doc2Vec models were trained on the training split only (100 dimensions, window 5, words seen at least 5 times, 20 epochs) with Gensim [8]. Training vectors stored during Doc2Vec training are not the same as vectors inferred later (mean cosine similarity 0.87 for PV-DM and 0.94 for PV-DBOW), so all three splits use inferred vectors, produced in the same way. Both new feature sets pass the Phase 2 leakage rule: the largest |r| between a text dimension and a label is 0.107 for PV-DM, 0.112 for PV-DBOW and 0.149 for Word2Vec, far below 0.85.

Table 1 compares the three text versions on the four models that ran on all of them. PV-DBOW and Word2Vec are level (mean ROC-AUC 0.856 and 0.855), and both beat PV-DM (0.801). This agrees with Lau and Baldwin, who found that the simpler PV-DBOW is the better Doc2Vec model [5]. PV-DM was therefore dropped, and every other model was trained on Word2Vec and PV-DBOW. For each model we report the feature set with the higher mean validation F1.

**Table 1:** The three text versions on linear regression, logistic regression, the perceptron and Naive Bayes (validation, mean over models and labels).

| Text features | Mean F1 | Mean ROC-AUC | Best toxicity F1 |
|---|---|---|---|
| Word2Vec (mean) | 0.121 | 0.855 | 0.352 |
| PV-DBOW | 0.124 | 0.856 | 0.349 |
| PV-DM | 0.101 | 0.801 | 0.313 |

### *2.2 Training and parameter choice*

**Parameter choice.** Every parameter is chosen on the validation split by F1 on that label, except where a model section says otherwise. The test split is used once, for the final numbers. Where a search on 1.78 million rows was too slow, it ran on a stratified sample (the same share of positives as the full data), and the best setting was refitted on all rows

when the model allowed it. The number of training rows is given for every model.

**Class imbalance.** Positives are rare: 8% for toxicity and under 1% for four labels. Most models use balanced class weights, so a mistake on a positive costs as much in total as all mistakes on negatives (Lab 2). A comment is predicted positive at the model's default cut: a probability of 0.5, or a decision value of 0.

**Metrics.** We record every Lecture 2 metric for every model, label and split: accuracy, error rate, precision, recall, specificity, false-positive rate (FPR), F1 and ROC-AUC. Accuracy is misleading on this data: answering "not toxic" for every comment is already 92% accurate. We compare models by F1, which balances precision and recall at the chosen cut, and by ROC-AUC, which measures how well the model ranks toxic comments above clean ones at every cut. A moderation queue uses both: the ranking decides what a moderator reads first, and the cut decides what is flagged.

**Hardware.** The light models ran on a laptop with 7.4 GB of RAM, reading the features in chunks from disk. The heavy models ran on a 16 GB laptop. All code uses scikit-learn [7].

## 3 THE MODELS

Table 2 summarises all models, Figure 1 shows F1 and ROC-AUC for every model and label, and Table 3 gives every metric on the test split. This section describes each model in the order of the lectures.

**Table 2:** Every model on the toxicity label, validation split unless marked. Mean = mean over the six labels. Time is the mean fit time per label.

| Model | Text | Train rows | Chosen setting (toxicity) | F1 | Prec. | Recall | AUC | Mean F1 | Mean AUC | Test F1 | Time |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Linear regression | PV-DBOW | 1,777,414 | balanced weights | 0.346 | 0.222 | 0.788 | 0.853 | 0.138 | 0.905 | 0.347 | 2 s |
| Logistic regression | Word2Vec | 1,777,414 | C = 0.01, balanced | 0.352 | 0.229 | 0.763 | 0.849 | 0.144 | 0.900 | 0.348 | 1.7 min |
| Perceptron | PV-DBOW | 1,777,414 | 3 passes, balanced | 0.275 | 0.171 | 0.704 | 0.775 | 0.106 | 0.822 | 0.278 | 3 s |
| Naive Bayes | PV-DBOW | 1,777,414 | learned priors | 0.304 | 0.200 | 0.632 | 0.752 | 0.112 | 0.792 | 0.304 | 4 s |
| ID3 | Word2Vec | 600,000 | depth 20, 28,333 nodes | 0.272 | 0.190 | 0.484 | 0.665 | 0.125 | 0.648 | 0.272 | 7.8 min |
| CART | Word2Vec | 600,000 | depth 20, 30,573 nodes | 0.294 | 0.207 | 0.504 | 0.675 | 0.132 | 0.682 | 0.295 | 3.7 min |
| C4.5 | Word2Vec | 1,777,414 | depth 20, ccp_alpha 5.4e-6, 53,957 nodes | 0.272 | 0.179 | 0.569 | 0.694 | 0.123 | 0.674 | 0.279 | 7.4 min |
| Random forest | Word2Vec | 300,000 | 100 trees, √features | 0.090 | 0.736 | 0.048 | 0.818 | 0.034 | 0.804 | 0.092 | 69 s |
| Linear SVM | Word2Vec | 1,777,414 | hinge, alpha 0.0001 | 0.345 | 0.222 | 0.767 | 0.846 | 0.139 | 0.891 | 0.338 | 56 s |
| RBF SVM | PV-DBOW | 50,000 | C = 1, gamma = scale | 0.409 | 0.331 | 0.534 | 0.851 | 0.227 | 0.886 | 0.407 | 6.1 min |
| MLP | Word2Vec | 1,777,414 | hidden (128, 64) | 0.368 | 0.703 | 0.250 | 0.886 | 0.198 | 0.919 | 0.367 | 1.6 min |
| AdaBoost | Word2Vec | 300,000 | 100 stumps | 0.210 | 0.577 | 0.128 | 0.828 | 0.111 | 0.873 | 0.212 | 7.4 min |
| KNN | Word2Vec | 150,000 | k = 3 | 0.235 | 0.363 | 0.174 | 0.655 | 0.120 | 0.560 | 0.235 | 27 s |

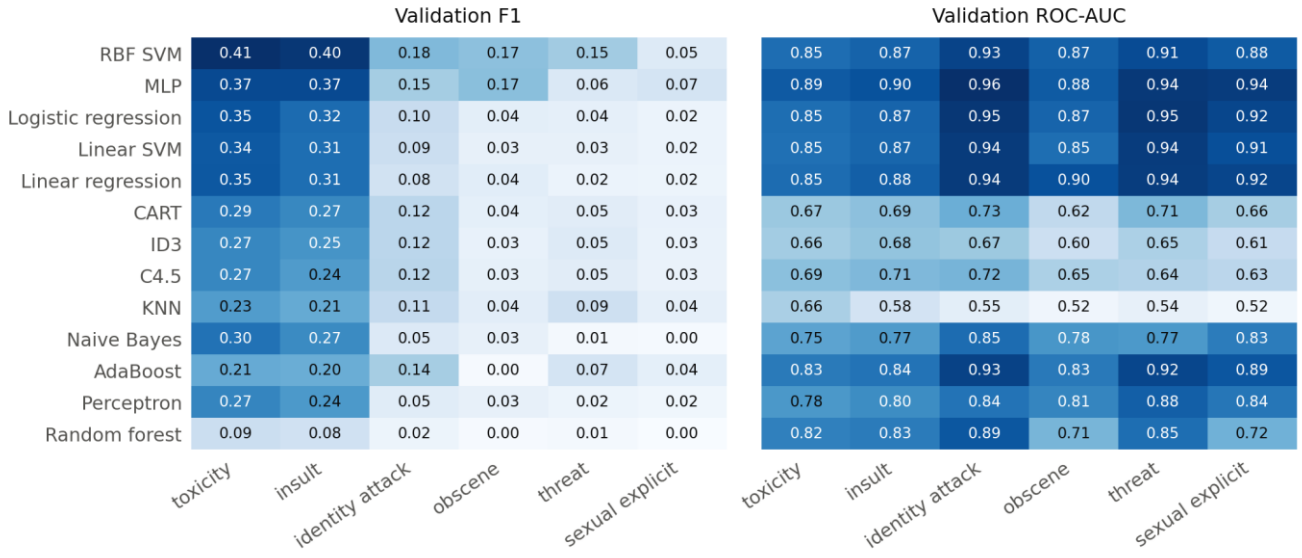



**Figure 1:** Validation F1 (left) and ROC-AUC (right) of every model on every label, each model on its better text features. Models are sorted by mean F1.

**Table 3:** Every Lecture 2 metric on the test split, toxicity label (each model on its better text features). FPR = false-positive rate.

| Model | Accuracy | Error rate | Precision | Recall | Specificity | FPR | F1 | ROC-AUC |
|---|---|---|---|---|---|---|---|---|
| Linear regression | 0.764 | 0.236 | 0.223 | 0.784 | 0.763 | 0.237 | 0.347 | 0.849 |
| Logistic regression | 0.775 | 0.225 | 0.226 | 0.750 | 0.777 | 0.223 | 0.348 | 0.844 |
| Perceptron | 0.709 | 0.291 | 0.173 | 0.700 | 0.710 | 0.290 | 0.278 | 0.770 |

| Model | Accuracy | Error rate | Precision | Recall | Specificity | FPR | F1 | ROC-AUC |
|---|---|---|---|---|---|---|---|---|
| Naive Bayes | 0.771 | 0.229 | 0.201 | 0.627 | 0.783 | 0.217 | 0.304 | 0.751 |
| ID3 | 0.796 | 0.204 | 0.190 | 0.477 | 0.824 | 0.176 | 0.272 | 0.661 |
| CART | 0.809 | 0.191 | 0.209 | 0.499 | 0.836 | 0.164 | 0.295 | 0.672 |
| C4.5 | 0.762 | 0.238 | 0.184 | 0.576 | 0.778 | 0.222 | 0.279 | 0.699 |
| Random forest | 0.923 | 0.077 | 0.732 | 0.049 | 0.998 | 0.002 | 0.092 | 0.812 |
| Linear SVM | 0.765 | 0.235 | 0.218 | 0.754 | 0.765 | 0.235 | 0.338 | 0.839 |
| RBF SVM | 0.876 | 0.124 | 0.330 | 0.531 | 0.906 | 0.094 | 0.407 | 0.852 |
| MLP | 0.931 | 0.069 | 0.692 | 0.249 | 0.990 | 0.010 | 0.367 | 0.882 |
| AdaBoost | 0.923 | 0.077 | 0.574 | 0.130 | 0.992 | 0.008 | 0.212 | 0.820 |
| KNN | 0.910 | 0.090 | 0.364 | 0.173 | 0.974 | 0.026 | 0.235 | 0.652 |

### *3.1 Linear regression (Lecture 4)*

**Setup.** Fitted by least squares on the 0/1 label, not on the raw score, and a comment is positive when the output is at least 0.5. This is a linear probability model, used as a classifier. Solved with the normal equations, accumulated in chunks over all 1,777,414 rows.

**Parameter choice.** No weights against balanced weights. Balanced weights won on 6 of 6 labels; without them, toxicity F1 is only 0.004, because few outputs reach 0.5 when 92% of the targets are 0.

**Performance.** Toxicity (PV-DBOW): F1 0.346, precision 0.222, recall 0.788, ROC-AUC 0.853. Mean over the six labels: F1 0.138, ROC-AUC 0.905. Test toxicity: F1 0.347, ROC-AUC 0.849.

**Comment.** A strong baseline: almost level with logistic regression in about 2 s per label. Its output is not a probability (it can fall below 0 or rise above 1), and the squared loss also punishes correct answers that are "too sure". It finds most toxic comments (recall 0.79) but flags many clean ones (precision 0.22).

### *3.2 Logistic regression (Lecture 5)*

**Setup.** Sigmoid output trained by maximum likelihood (L-BFGS), L2 regularisation, balanced class weights, all 1,777,414 rows.

**Parameter choice.** The inverse regularisation strength C from {0.01, 0.1, 1, 10, 100}, chosen on a 200,000-row stratified sample and refitted on all rows. C = 0.01, the strongest regularisation, won on 5 of 6 labels.

**Performance.** Toxicity (Word2Vec): F1 0.352, precision 0.229, recall 0.763, ROC-AUC 0.849. Mean over the six labels: F1 0.144, ROC-AUC 0.900. Test toxicity: F1 0.348, ROC-AUC 0.844.

**Comment.** The best linear model. It outputs a probability, so the cut can be moved without retraining, and its weights show which features push a comment towards toxic. It trains in about 1.7 min per label. Strong regularisation winning means that more freedom does not help: the limit is the straight-line boundary, not the amount of data.

### *3.3 Perceptron (Lecture 6)*

**Setup.** Rosenblatt's update rule with balanced class weights, all rows, trained in chunks of 100,000 rows.

**Parameter choice.** The number of passes over the data, up to 10, stopping after 2 passes without a better validation F1. Between 1 and 3 passes were kept.

**Performance.** Toxicity (PV-DBOW): F1 0.275, precision 0.171, recall 0.704, ROC-AUC 0.775. Mean over the six labels: F1 0.106, ROC-AUC 0.822. Test toxicity: F1 0.278, ROC-AUC 0.770.

**Comment.** Fast (about 3 s per label) but the weakest linear model. It stops updating once a point is on the right side, so it finds no margin, and when the classes overlap, as here, its weights keep jumping: validation F1 went up and down from pass to pass. It gives no probability.

### *3.4 Naive Bayes (Lecture 11)*

**Setup.** Gaussian Naive Bayes: one normal distribution per feature and class, all rows.

**Parameter choice.** Class priors learned from the data against uniform priors (0.5/0.5). Learned priors won on 6 of 6 labels.

**Performance.** Toxicity (PV-DBOW): F1 0.304, precision 0.200, recall 0.632, ROC-AUC 0.752. Mean over the six labels: F1 0.112, ROC-AUC 0.792. Test toxicity: F1 0.304, ROC-AUC 0.751.

**Comment.** The fastest model and a useful baseline. Its assumption that features are independent given the label is wrong here: the 100 text dimensions describe one comment together, and the count features are skewed rather than normal. On the rare labels it flags far too many comments (37,044 for threat, which has 224 positives in validation), because multiplying 133 densities makes its probabilities extreme.

### *3.5 Decision tree: ID3 (Lecture 9)*

**Setup.** Splits chosen by information gain (entropy), no pruning, balanced class weights. scikit-learn has no true ID3: it splits a continuous feature at a threshold, which ID3 itself cannot do without first binning the feature (Lecture 10). The final tree was fitted on a 600,000-row stratified sample, because the full data did not fit in memory.

**Parameter choice.** Maximum depth from {5, 10, 15, 20}, chosen on a 200,000-row sample. Depth 20 won for toxicity (28,333 nodes).

**Performance.** Toxicity (Word2Vec): F1 0.272, precision 0.190, recall 0.484, ROC-AUC 0.665. Mean over the six labels: F1 0.125, ROC-AUC 0.648. Test toxicity: F1 0.272, ROC-AUC 0.661.

**Comment.** Depth 20, the deepest setting, won on 6 of 6 labels. Its F1 is close to the other trees (mean 0.125, against 0.132 for CART), so entropy and Gini lead to much the same splits, but without pruning it ranks comments worst of the three trees (mean ROC-AUC 0.648). The tree fits its training rows far better than new ones (toxicity F1 0.410 on training, 0.272 on validation). Its rules are readable in principle, but a tree of 28,333 nodes over embedding dimensions is not something a moderator can read.

### *3.6 Decision tree: CART (Lecture 10)*

**Setup.** Binary splits chosen by Gini impurity, balanced class weights. The final tree was fitted on a 600,000-row stratified sample, because the full data did not fit in memory.

**Parameter choice.** Maximum depth from {5, 10, 15, 20}, chosen on a 200,000-row sample. Depth 20 won for toxicity (30,573 nodes).

**Performance.** Toxicity (Word2Vec): F1 0.294, precision 0.207, recall 0.504, ROC-AUC 0.675. Mean over the six labels: F1 0.132, ROC-AUC 0.682. Test toxicity: F1 0.295, ROC-AUC 0.672.

**Comment.** Validation F1 rose with every extra level, and depth 20, the deepest setting tried, won on 6 of 6 labels, so the tree is still short of what the data could support. It is slightly better than C4.5 (mean F1 0.132 against 0.123) while trained on fewer rows, but it has the same limits: one feature per split, a ranking well below the linear models (mean ROC-AUC 0.682), and a large gap between training and validation F1 on toxicity (0.439 against 0.294).

### *3.7 Decision tree: C4.5 (Lecture 10)*

**Setup.** Entropy splits, maximum depth 20, balanced class weights, and pruning chosen on the validation split; final tree on all rows. scikit-learn has no gain ratio or rule post-pruning, so cost-complexity pruning stands in for C4.5's pruning.

**Parameter choice.** The pruning strength ccp_alpha: up to 20 values from the tree's pruning path, chosen by validation accuracy on a 300,000-row sample.

**Performance.** Toxicity (Word2Vec): F1 0.272, precision 0.179, recall 0.569, ROC-AUC 0.694. Mean over the six labels: F1 0.123, ROC-AUC 0.674. Test toxicity: F1 0.279, ROC-AUC 0.699.

**Comment.** Easy to read as rules, but weak here. Each split looks at one feature, while the text signal is spread over 100 dimensions that only mean something together. Choosing the pruning by accuracy favoured almost no pruning (accuracy barely changes on an imbalanced label), so the toxicity tree has 53,957 nodes and fits its training data far better (F1 0.478) than new comments.

### *3.8 Random forest (Lecture 10)*

**Setup.** Bagged, fully grown trees with √133 features tried per split and class weights balanced inside each bootstrap sample, on a 300,000-row stratified sample.

**Parameter choice.** The number of trees from {100, 200, 300}: 4 labels chose 100 trees and 2 chose 200.

**Performance.** Toxicity (Word2Vec): F1 0.090, precision 0.736, recall 0.048, ROC-AUC 0.818. Mean over the six labels: F1 0.034, ROC-AUC 0.804. Test toxicity: F1 0.092, ROC-AUC 0.812.

**Comment.** Its ROC-AUC (0.804 on average) shows that it ranks comments sensibly, but at the 0.5 cut it almost never says "toxic": it flags 497 of 97,320 validation comments for toxicity, and at most 1 for obscene, threat and sexual_explicit. Fully grown trees memorise their bootstrap sample (training F1 1.00), so their votes on a new rare positive rarely reach a majority, a known weakness of forests on imbalanced data [15]. A lower cut would recover much of this. Slow to train and hard to interpret.

### *3.9 Linear SVM (Lecture 12)*

**Setup.** Hinge loss with L2 regularisation, trained by stochastic gradient descent on all rows (an exact SVM solver is far too slow at 1.78 million rows), balanced class weights.

**Parameter choice.** Fixed, not searched: regularisation alpha = 0.0001, scikit-learn's default.

**Performance.** Toxicity (Word2Vec): F1 0.345, precision 0.222, recall 0.767, ROC-AUC 0.846. Mean over the six labels: F1 0.139, ROC-AUC 0.891. Test toxicity: F1 0.338, ROC-AUC 0.839.

**Comment.** Level with logistic regression, as expected: both draw a straight boundary and differ only in the loss. It gives a decision value instead of a probability and adds nothing over logistic regression here.

### *3.10 Kernel SVM with RBF kernel (Lecture 12)*

**Setup.** Soft-margin SVM with a Gaussian (RBF) kernel and balanced class weights, on a 50,000-row stratified sample (2.8% of the training data). An exact kernel SVM costs at least the square of the number of rows to train [12], and one fit on 50,000 rows already took 15 min for toxicity.

**Parameter choice.** Fixed, not searched: C = 1 and gamma = 1 / (features × feature variance), scikit-learn's "scale" setting. A grid search was too costly at this size; Section 4 measures the effect of the sample instead.

**Performance.** Toxicity (PV-DBOW): F1 0.409, precision 0.331, recall 0.534, ROC-AUC 0.851. Mean over the six labels: F1 0.227, ROC-AUC 0.886. Test toxicity: F1 0.407, ROC-AUC 0.852.

**Comment.** The best F1 of all models on 4 of 6 labels (toxicity, insult, identity attack, threat), although it saw only 2.8% of the data and was not tuned. Its curved boundary fits the overlap between toxic and clean comments better than a straight line, and balanced weights put its default cut close to the best one (Section 4). The costs: training time grows quickly with rows, 23,176 support vectors must be stored and compared with every new comment, there is no probability, and it is hard to explain. Its training F1 (0.73) is far above validation, a sign that 50,000 rows are too few for the boundary it draws.

### *3.11 Neural network: MLP (Lecture 7)*

**Setup.** A feed-forward network with ReLU hidden layers and a sigmoid output, trained by back-propagation with the Adam optimiser and early stopping on 10% of the training rows, refitted on all rows. scikit-learn's MLP has no class weights, so it learns the real class balance.

**Parameter choice.** Hidden layers from {(64), (128), (128, 64)}, chosen on a 300,000-row sample. Two layers (128, 64) won on 2 labels, including toxicity; one layer won on the others.

**Performance.** Toxicity (Word2Vec): F1 0.368, precision 0.703, recall 0.250, ROC-AUC 0.886. Mean over the six labels: F1 0.198, ROC-AUC 0.919. Test toxicity: F1 0.367, ROC-AUC 0.882.

**Comment.** The best ranking of all models: the highest ROC-AUC on 4 of 6 labels. Without class weights it is cautious: 70% of its "toxic" flags are right, but it finds only 25% of toxic comments. Its output is a probability, so a lower cut trades precision for recall without retraining (Section 4). It trains in about 1.6 min per label on all rows, but its weights cannot be read as reasons.

### *3.12 AdaBoost (Lecture 13)*

**Setup.** 100 decision stumps (one-split trees), boosted, on a 300,000-row stratified sample, no class weights.

**Parameter choice.** Fixed: 100 stumps, the setting of the Lecture 13 example.

**Performance.** Toxicity (Word2Vec): F1 0.210, precision 0.577, recall 0.128, ROC-AUC 0.828. Mean over the six labels: F1 0.111, ROC-AUC 0.873. Test toxicity: F1 0.212, ROC-AUC 0.820.

**Comment.** Good ranking (mean ROC-AUC 0.873) but low recall at the default cut (0.13 for toxicity), because without class weights the boosted vote stays on the majority class. Each stump looks at one feature, which suits the text dimensions poorly. Training is sequential and took about 7.4 min per label.

### *3.13 K-nearest neighbours (Lab 1, Assignment 1)*

**Setup.** Euclidean distance, majority vote, on a 150,000-row stratified sample, no class weights.

**Parameter choice.** k from {1, 3, 5, 9, 15, 21}. k = 1 won on 5 of 6 labels and k = 3 on toxicity: a larger k lets the majority class outvote the rare positives.

**Performance.** Toxicity (Word2Vec): F1 0.235, precision 0.363, recall 0.174, ROC-AUC 0.655. Mean over the six labels: F1 0.120, ROC-AUC 0.560. Test toxicity: F1 0.235, ROC-AUC 0.652.

**Comment.** The weakest ranking (mean ROC-AUC 0.560, close to chance on the rare labels). In 133 dimensions the distances from a comment to its nearest and its farthest neighbours become similar, so the nearest neighbours carry little signal [14]. Prediction is slow, since every new comment is compared with every stored one. Not a good fit.

### *3.14 Bayesian network (Lecture 11): not run*

A Bayesian network needs a known structure, built from causal relationships or domain knowledge, and it stores a probability table for each node. Our 133 features do not fit this. The text dimensions and the scaled statistics are continuous, and embedding dimensions have no causal meaning, so no structure can be drawn from domain knowledge. Learning a structure without that knowledge depends heavily on the order of the features, and the probability tables would first require binning every feature, which discards information. Naive Bayes, the special case whose structure treats every feature as independent given the label, was run instead and represents this family.

## 4 THE TOP THREE MODELS

### *4.1 Choosing the three*

We rank the models by both headline metrics, averaged over the six labels (Table 2). The RBF SVM has the highest mean F1 (0.227), the MLP the second-highest F1 (0.198) and the highest mean ROC-AUC (0.919). Third place is close: logistic regression, linear regression and the linear SVM are within 0.007 of each other on toxicity F1. We take logistic regression, which has the highest mean F1 of the three (0.144) and, unlike the other two, outputs a real probability. The three also represent three different families: a kernel method, a neural network and a linear model. The rest of this section uses each model on its better text features: PV-DBOW for the SVM and Word2Vec for the other two.

### *4.2 Head-to-head on the test split*

Table 4 gives test F1 and ROC-AUC for every label. At the default cut the SVM has the best toxicity F1 and the MLP the best ROC-AUC. To check that these gaps are not luck of the test sample, we redrew the 97,320 test comments 1,000 times with replacement and recomputed both metrics (a bootstrap). The 95% intervals for toxicity F1 are 0.398–0.416 for the SVM, 0.363–0.386 for the MLP and 0.341–0.354 for logistic regression, and for ROC-AUC 0.879–0.886 (MLP), 0.848–0.856 (SVM) and 0.839–0.848 (logistic regression). The F1 intervals do not overlap, and the MLP's ROC-AUC interval lies clear of the other two (those of the SVM and logistic regression touch), so the order is not an accident of the test sample.

Dietterich recommends McNemar's test for comparing two classifiers that can be trained only once [11]. It counts the comments that one model gets right and the other gets wrong. The MLP is right where the SVM is wrong on 7,948 comments and the reverse happens on 2,624 ($\chi^2 = 2{,}680$, far above 3.84, the 5% level). This measures errors of either kind, so it rewards the MLP for flagging fewer comments: it says the MLP is more often right, not that it finds more toxic comments. The two metrics we care about disagree, which is the first sign that the comparison depends on how the models are used.

**Table 4:** Test F1 and ROC-AUC of the top three on every label, at each model's default cut.

| Label | SVM F1 | MLP F1 | LogReg F1 | SVM AUC | MLP AUC | LogReg AUC |
|---|---|---|---|---|---|---|
| toxicity | 0.407 | 0.367 | 0.348 | 0.852 | 0.882 | 0.844 |
| insult | 0.401 | 0.374 | 0.311 | 0.869 | 0.898 | 0.867 |
| identity attack | 0.179 | 0.100 | 0.102 | 0.924 | 0.961 | 0.946 |
| obscene | 0.148 | 0.137 | 0.041 | 0.884 | 0.885 | 0.864 |
| threat | 0.129 | 0.077 | 0.036 | 0.917 | 0.959 | 0.952 |
| sexual explicit | 0.039 | 0.083 | 0.029 | 0.903 | 0.940 | 0.914 |

### *4.3 Variability*

A single number per model hides how much it would move if the experiment were repeated. Bouthillier et al. list the main sources of this variance: which rows are sampled for training and testing, the random initialisation, and the choice of settings, and find that the data sample matters most [10]. We measured each source that applies to our three models. Table 5 summarises the results; this section explains them.

**Data sample: cross-validation.** To compare the models on equal terms, we drew a stratified pool of 62,500 training comments per label and ran 5-fold cross-validation, with the same folds for all three models, so each model trains on 50,000 rows and is tested on the remaining fifth (Figure 2). On toxicity, F1 is 0.407 ± 0.016 for the SVM, 0.273 ± 0.019 for the MLP and 0.351 ± 0.005 for logistic regression (mean ± standard deviation over folds). Logistic regression barely moves from fold to fold: it has only 134 weights and a convex loss, so any large sample leads to almost the same model. The SVM and MLP move about three times as much.

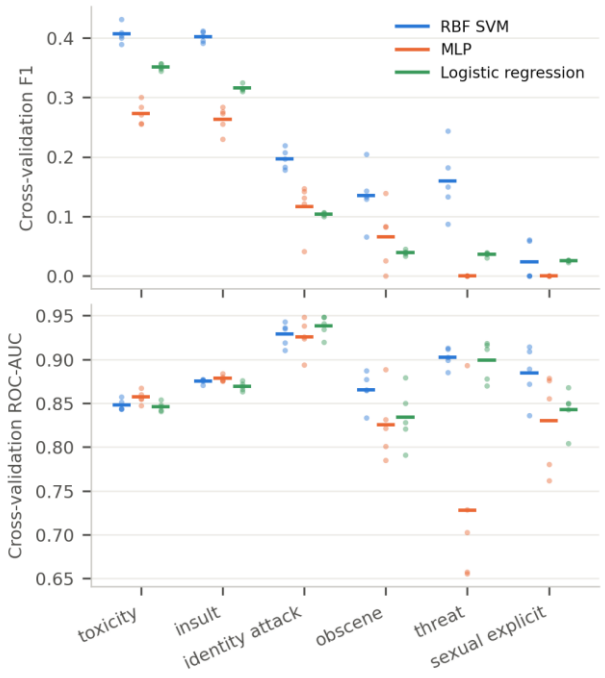



**Figure 2:** F1 (top) and ROC-AUC (bottom) of the top three in 5-fold cross-validation on 62,500 training comments per label. Dots are folds; bars are means.

The spread grows sharply on the rare labels. For threat, SVM F1 ranges from 0.087 to 0.244 across folds. The reason is the number of positives: the pool holds 149 threat comments, so each test fold has about 30, and a few comments flagged correctly or not change F1 a lot. The same holds for ROC-AUC, which also needs positives to rank. The validation split has 224 threat positives, so its numbers are steadier, but F1 below 0.2 on the four rare labels should be read as a rough figure for every model.

**Training rows.** The cross-validation also isolates the effect of data size, since every model there saw 50,000 rows. On toxicity, logistic regression scores the same with 50,000 rows as with all 1,777,414 (F1 0.351 against 0.352): a straight boundary is already fully learned from a small sample. The MLP gains the most from data: its toxicity F1 rises from 0.273 to 0.368, and its ROC-AUC from 0.857 to 0.886. On 50,000 rows it never flags a threat or sexual_explicit comment (F1 0.000), because without class weights the 119 or so positives it sees are outweighed by the negatives. At equal size the SVM has the best F1 on every label but sexual explicit and ranks toxicity about as well as the MLP (ROC-AUC 0.848 against 0.857). In other words, the MLP's lead in ranking comes from using all the data, which the SVM cannot do [12].

**Random seed and sample.** We retrained each model five times on toxicity and threat, evaluated on validation. The MLP was refitted on all rows with five different seeds, which change its starting weights and the 10% of rows held out for early stopping. The SVM was refitted on five different 50,000-row samples. Toxicity F1 is 0.407 ± 0.004 for the SVM and 0.375 ± 0.013 for the MLP (range 0.358–0.392), while the MLP's ROC-AUC hardly moves (0.886 ± 0.000). The seed changes where the MLP's scores fall relative to the 0.5 cut more than how it orders comments, so F1 moves and ROC-AUC does not. On threat, F1 varies much more for both (SVM 0.141 ± 0.017, MLP 0.088 ± 0.028). Logistic regression has a single optimum, so its seed has no effect, and it was not repeated.

**Text features.** The choice between Word2Vec and PV-DBOW matters little on the common labels and a lot on the rare ones. The SVM's toxicity F1 is 0.409 on PV-DBOW and 0.392 on Word2Vec, but on identity attack the order flips (0.183 against 0.253), as it does on sexual explicit (0.053 against 0.127). A 50,000-row sample holds only about 131 sexual explicit positives, and these labels are the noisiest in the cross-validation too (sexual explicit F1 0.000–0.061 across folds). One run per feature set cannot separate a real preference from this noise, so we keep one feature set per model.

**Label threshold.** Phase 1 committed to repeating the evaluation with the label defined at 0.4 and 0.6 instead of 0.5. We rebuilt the toxicity label at each threshold and retrained the three models (Table 5). A lower threshold makes positives more common (11.2% of training comments at 0.4, 5.3% at 0.6), and every model's F1 rises with it: the SVM's goes from 0.351 at 0.6 to 0.453 at 0.4. ROC-AUC moves the other way, because at 0.6 the positives are the comments most raters agreed on, which are easier to rank. The order of the three models is the same at every threshold, so the comparison does not depend on where the label line is drawn.

**Decision cut.** The largest source of variation is the cut that turns a score into a yes or no. Figure 3 shows validation F1 for every cut, expressed as the share of comments flagged. Each model's default cut sits at a different point on its curve. The SVM, trained with balanced weights, flags 12.7% of comments and is already near its best F1 (0.409 against a best of 0.414). Logistic regression, also balanced, flags 26.3%, too many. The MLP, trained without weights, flags only 3.0%, far too few. This analysis uses a refitted copy of each model, so its default-cut numbers differ slightly from Table 2. Choosing the cut on validation and applying it once to test gives test F1 0.487 for the MLP, 0.415 for logistic regression and 0.410 for the SVM. The order of Section 4.2 changes: with a tuned cut, the MLP is clearly the best classifier, as its higher ROC-AUC predicted.

The MLP's best cut, 0.243, is almost exactly half of its best F1 (0.485). Lipton et al. prove that this holds for a model whose scores are calibrated probabilities [9]. The MLP was trained on the real class balance, so its scores behave like probabilities; the balanced weights of logistic regression inflate its scores, which moves its best cut up to 0.721. Saito and Rehmsmeier warn that on imbalanced data ROC-AUC can look good while precision is poor [13]; Figure 3 is the check on that, and it agrees with ROC-AUC here.

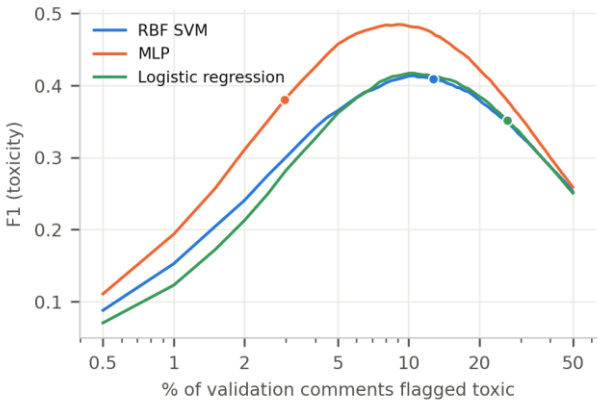



**Figure 3:** Validation F1 on toxicity as the decision cut moves, shown as the share of comments flagged toxic (log scale). Dots mark each model's default cut: probability 0.5 for the MLP and logistic regression, decision value 0 for the SVM.

**Identity groups.** Phase 1 also committed to checking whether the models treat comments about particular groups unfairly, with the metrics of Borkan et al. [3]. For each identity mentioned in at least 100 test comments (13 identities), we compute three ROC-AUCs on toxicity: inside the group (subgroup AUC); toxic comments outside the group against clean comments that mention it (BPSN, where a low value means clean comments about the group are scored as toxic); and the reverse (BNSP, where a low value means toxic comments about the group are missed). Figure 4 shows the results. The MLP has the highest subgroup AUC for all 13 identities and the highest BNSP for 13; its BPSN is highest for 9 and the SVM's for the other 4.

All three models share the same weakness. Their BPSN is below their BNSP for nearly every group (10 of 13 for the RBF SVM, 13 of 13 for the MLP, 13 of 13 for logistic regression), and the lowest BPSN is always for the same group, "white" (0.688, 0.665, 0.620). That is, a clean comment that mentions a group is ranked above toxic comments elsewhere too often. The cause is in the data: 30.0% of test comments that mention white people and 32.3% of those that mention black people are toxic, against 8.0% overall, so the identity words themselves become evidence of toxicity, exactly the bias Borkan et al. describe [3]. Logistic regression, which can only add up feature effects, shows it most (BPSN 0.620 for "white"). The MLP reduces the problem but does not remove it.

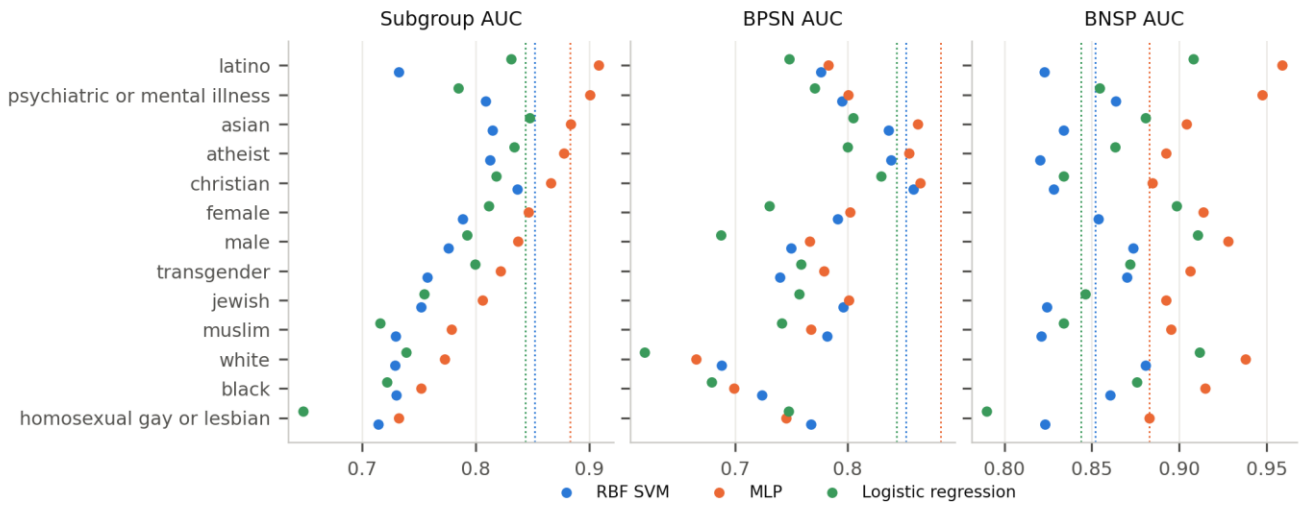



**Figure 4:** Bias metrics on the test split for the 13 identities mentioned in at least 100 test comments, toxicity label. Low BPSN: clean comments that mention the group are ranked as toxic; low BNSP: toxic comments that mention it are missed. Dotted lines mark each model's ROC-AUC on all test comments.

**Validation against test.** Every parameter and cut was chosen on validation, so test is the honest check. Test F1 differs from validation F1 by at most 0.004 on toxicity for the three models, and the tuned cuts carry over (MLP: 0.485 on validation, 0.487 on test). The large, well-mixed splits mean that choosing on validation did not over-fit it.

**Table 5:** How much each result moves, toxicity label unless stated. ± is the standard deviation over folds or runs. The label-threshold and cut rows use retrained copies of each model, so they can differ slightly from Table 2.

| Source | RBF SVM | MLP | LogReg |
|---|---|---|---|
| CV folds (F1) | 0.407 ± 0.016 | 0.273 ± 0.019 | 0.351 ± 0.005 |
| CV folds, threat (F1) | 0.159 ± 0.058 | 0.000 ± 0.000 | 0.036 ± 0.003 |
| Seeds / samples (F1) | 0.407 ± 0.004 | 0.375 ± 0.013 | none (one optimum) |
| Rows: 50k → all (F1) | 50k only | 0.273 → 0.368 | 0.351 → 0.352 |
| Label at 0.4 / 0.5 / 0.6 (F1) | 0.45 / 0.41 / 0.35 | 0.43 / 0.38 / 0.32 | 0.41 / 0.35 / 0.28 |
| Default → best cut (test F1) | 0.407 → 0.410 | 0.375 → 0.487 | 0.348 → 0.415 |
| Subgroup AUC (min–max) | 0.71–0.84 | 0.73–0.91 | 0.65–0.85 |
| Validation → test (F1) | 0.409 → 0.407 | 0.368 → 0.367 | 0.352 → 0.348 |

### *4.4 Why the results vary*

- **Too few positives.** On the four rare labels, a fold or sample holds tens to a few hundred positives, so F1 and ROC-AUC are noisy for every model. This is a property of the data, not of any model.
- **Model capacity.** Logistic regression has few parameters and one optimum, so it is stable but cannot improve with more data. The SVM and MLP can draw curved boundaries, which helps but also lets them follow the particular rows they saw, so they vary more.
- **Training rows.** The SVM is limited to 50,000 rows by its cost; the MLP uses all 1.78 million. Comparisons at the default setup therefore mix model quality with data quantity.
- **Class weights and the cut.** Balanced weights shift scores up, no weights keep them low. Most of the F1 difference between the models at the default cut is a difference in how many comments each one flags, not in how well it ranks them.
- **Biased signal.** Identity words are more common in toxic comments, so all models over-flag clean comments that mention some groups.

## 5 CHOICE OF TECHNIQUE

We choose the **MLP** for the rest of the project, weighing three things.

**Performance.** The MLP ranks comments best: among the top three it has the highest test ROC-AUC on 6 of 6 labels (Table 4) and the highest subgroup AUC for every one of the 13 identity groups. Its lower F1 at the 0.5 cut comes from flagging too few comments, not from ranking them badly. With a cut chosen on validation, its test F1 on toxicity is 0.487, well above the SVM (0.410) and logistic regression (0.415), and the cut carried over from validation to test unchanged (Section 4.3).

**Nature of the model.** The MLP learns curved boundaries, which the text embeddings need, and it is trained by gradient descent in small batches, so it scales to all 1,777,414 comments in about 1.6 min per label. The RBF SVM cannot: its cost grows at least with the square of the rows [12], so it was limited to 50,000 rows, and at that size the MLP had lower F1 than the SVM on every label in cross-validation. The MLP's gain from more data (toxicity F1 0.273 on 50,000 rows, 0.368 on all) also suggests that it can still improve. Its scores behave like probabilities (its best cut is half its best F1 [9]), so a single cut has a clear meaning. Logistic regression has the same property and is more stable, but its straight boundary caps it, and it shows the strongest identity bias.

**Fit to the problem.** The moderation aid sorts comments so that a moderator reads the worst first, and flags those above a cut that the moderator can move to match the workload. That calls for a good ranking and a probability-like score, which is what the MLP provides. The task is also multi-label (Phase 1): one network with six sigmoid outputs could share its hidden layers between the attributes, which may help the rare labels.

**Weaknesses to address.** The MLP's F1 moves with its seed (toxicity 0.375 ± 0.013), it has no class weights, and its weights cannot be read as reasons. For the next phase we will choose the cut per label on validation, average several seeds,

give rare positives more weight by oversampling them in training, and keep checking the identity-group metrics, since the MLP reduces the bias on mentions of groups such as "white" and "black" but does not remove it. Logistic regression stays as the interpretable baseline.

## 6 CONCLUSION

We trained 13 supervised models from the course on the same 1,777,414 training comments and 133 features, and explained why the Bayesian network was not used. Models that combine the embedding dimensions (the SVMs, the MLP, linear and logistic regression) clearly beat models that split on one feature at a time (the trees, AdaBoost) or rely on distances (KNN). Comparing the top three in depth showed that the headline F1 depends as much on each model's default cut and on the number of rows it can use as on the model itself, and that rare labels give noisy results for every model. Measured fairly, the MLP is the strongest and most suitable technique, and it is our choice for the final phase.

## AI USAGE DISCLOSURE

This report was completed with assistance from Claude (Anthropic). The AI was used to write analysis, and feature-generation scripts, to derive the statistics and draw the figures from their outputs. All numbers in the report are read from the outputs of the submitted scripts, the decisions were reviewed by the authors, and the authors are accountable for the accuracy of the content.

## REFERENCES


[1] Moaz Allam and Kareem Elhenawy. 2026. Detecting Toxic Comments in Online Discussions: A Literature Review and Dataset Survey. CSCE 3602 / DSCI 3415 class project, Phase 1 report. The American University in Cairo.

[2] Moaz Allam and Kareem Elhenawy. 2026. Detecting Toxic Comments in Online Discussions: Preparing the Civil Comments Data. CSCE 3602 / DSCI 3415 class project, Phase 2 report. The American University in Cairo.

[3] Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2019. Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. In Companion Proceedings of The 2019 World Wide Web Conference. arXiv:1903.04561

[4] Quoc Le and Tomas Mikolov. 2014. Distributed Representations of Sentences and Documents. In Proceedings of the 31st International Conference on Machine Learning (ICML). PMLR 32(2), 1188-1196.

[5] Jey Han Lau and Timothy Baldwin. 2016. An Empirical Evaluation of doc2vec with Practical Insights into Document Embedding Generation. In Proceedings of the 1st Workshop on Representation Learning for NLP. ACL, 78-86.

[6] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781

[7] Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825-2830.

[8] Radim Řehůřek and Petr Sojka. 2010. Software Framework for Topic Modelling with Large Corpora. In Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks. ELRA, 45-50.

[9] Zachary C. Lipton, Charles Elkan, and Balakrishnan Naryanaswamy. 2014. Optimal Thresholding of Classifiers to Maximize F1 Measure. In Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2014). Springer, 225-239.

[10] Xavier Bouthillier et al. 2021. Accounting for Variance in Machine Learning Benchmarks. In Proceedings of Machine Learning and Systems 3 (MLSys 2021). arXiv:2103.03098

[11] Thomas G. Dietterich. 1998. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. Neural Computation 10, 7, 1895-1923.

[12] Léon Bottou and Chih-Jen Lin. 2007. Support Vector Machine Solvers. In Large-Scale Kernel Machines, L. Bottou, O. Chapelle, D. DeCoste, and J. Weston (Eds.). MIT Press, 1-27.

[13] Takaya Saito and Marc Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE 10, 3, e0118432.

[14] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. 1999. When Is "Nearest Neighbor" Meaningful? In Database Theory (ICDT'99). LNCS 1540, Springer, 217-235.

[15] Chao Chen, Andy Liaw, and Leo Breiman. 2004. Using Random Forest to Learn Imbalanced Data. Technical Report 666, Department of Statistics, University of California, Berkeley.